

Datenlayout: Data Skipping, Z-Ordering, Partitionierung

Section 6 · Troubleshooting, Monitoring and Optimization · Quelle: Kurs 7, Kapitel 2.2.1–2.2.4

Wie schnell eine Abfrage auf einer Delta-Tabelle läuft, hängt maßgeblich davon ab, wie die zugrunde liegenden Dateien physisch organisiert sind. Dieses Kapitel behandelt drei klassische Techniken zur Optimierung des **Datenlayouts**: Data Skipping, Z-Ordering und Partitionierung – sowie die Statistiken im Transaktionsprotokoll, auf denen sie beruhen. Sie bilden die konzeptionelle Grundlage für Liquid Clustering, das in Kapitel 06-3 behandelt wird und diese Techniken bei neuen Tabellen inzwischen weitgehend ablöst.

1. Data Skipping: Dateien ungelesen überspringen

Data Skipping ist eine Optimierung, bei der Spark Dateien (oder Teile davon) gar nicht erst öffnet, wenn feststeht, dass sie keine für die Abfrage relevanten Daten enthalten können. Möglich wird das durch Statistiken, die Delta Lake beim Schreiben pro Datei sammelt – typischerweise Minimum- und Maximum-Werte, Null-Zähler und Zeilenzahlen je Spalte. Diese Statistiken liegen nicht in den Dateien selbst, sondern im **Transaktionsprotokoll** (Delta Log). Enthält eine Abfrage einen Filter wie `WHERE datum = '2026-07-01'`, prüft Spark zunächst die Metadaten: Überschneidet sich der Min/Max-Bereich einer Datei nicht mit dem gesuchten Wert, wird die Datei komplett übersprungen – ganz ohne Festplattenzugriff.

```
-- Minimal- und Maximalwerte je Datei ermitteln (illustriert das Prinzip hinter Data
Skipping)
SELECT
    input_file_name() AS file_name,
    min(col) AS col_min,
    max(col) AS col_max
FROM table
GROUP BY input_file_name();
```

Der Nutzen ist unmittelbar einleuchtend: weniger I/O bedeutet schnellere Abfragen, insbesondere bei großen Tabellen mit vielen Dateien. Data Skipping funktioniert am besten, wenn die Daten physisch nach den Spalten geclustert sind, auf die häufig gefiltert wird – genau hier setzen Z-Ordering und Partitionierung an.

2. Statistiken im Transaktionsprotokoll

Damit Data Skipping und Z-Ordering überhaupt funktionieren, muss Delta Lake zunächst Statistiken sammeln. Standardmäßig erfasst Databricks Delta Lake Min/Max-Werte für die **ersten 32 Spalten** einer Tabelle (Konfigurationsparameter `dataSkippingNumIndexedCols`). Manche Abfragen – etwa `SELECT max(col) FROM table` – lassen sich sogar rein aus den Metadaten beantworten, ganz ohne dass eine einzige Datenzeile gelesen werden muss.

Beim Filtern wendet Spark die verschiedenen Filterarten in einer festen Reihenfolge an: zuerst Partitionsfilter, dann Datenfilter (Data Skipping), zuletzt Pushed Filters. Zwei Einschränkungen sind zu beachten:

- Bei **Timestamp**- und **String**-Spalten können Präzisions- oder Trunkierungseffekte dazu führen, dass die Statistiken keine exakten Treffer liefern – Spark muss dann teilweise doch auf einen vollständigen Dateiscan zurückfallen.
- Für Spalten mit sehr langen Strings sollten keine Statistiken gesammelt werden, da sie den Metadaten-Overhead unnötig aufblähen. Entweder werden solche Spalten hinter die ersten 32 Spalten verschoben (`ALTER TABLE ... CHANGE COLUMN ... AFTER col32`), oder die Anzahl indizierter Spalten wird direkt reduziert:

```
SET spark.databricks.delta.properties.defaults.dataSkippingNumIndexedCols = 3;
```

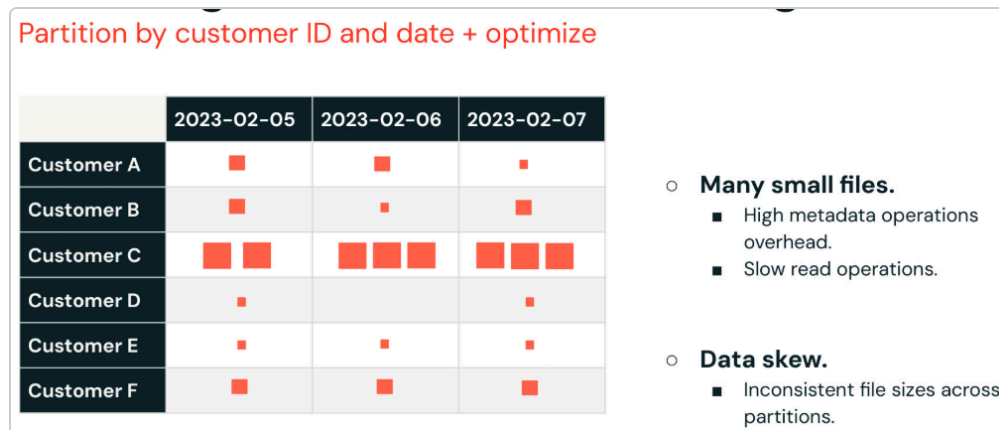
3. Z-Ordering: Ähnliche Werte physisch zusammen ablegen

Z-Ordering organisiert die Daten einer Tabelle anhand einer oder mehrerer Spalten so, dass ähnliche Werte in denselben Dateien landen. Dabei wird jede Datei anschließend mit ihren Min/Max-Werten für die Z-Order-Spalte(n) im Dateifußbereich (z. B. bei Parquet) versehen. Filtert eine Abfrage nun auf die Z-Order-Spalte, kann Spark anhand dieser Statistiken gezielt genau die eine relevante Datei öffnen, statt mehrere Dateien "auf Verdacht" zu lesen. Der Effekt ist besonders stark bei Spalten mit vielen unterschiedlichen Werten (hoher Kardinalität).

Databricks empfiehlt inzwischen grundsätzlich **Liquid Clustering** anstelle von Z-Ordering für neue Tabellen (siehe Kapitel 06-3) – Z-Ordering bleibt aber weiterhin unterstützt und in bestehenden Umgebungen relevant.

4. Partitionierung: Nutzen und Risiken

Partitionierung ist die klassischste Layout-Technik: Daten werden anhand einer Spalte in physisch getrennte Verzeichnisse aufgeteilt, sodass Spark ganze Partitionen überspringen kann, statt einzelne Dateien zu prüfen. Databricks rät jedoch generell von Partitionierung ab, da sie in der Praxis häufig falsch eingesetzt wird – typische Folgen sind eine **Explosion kleiner Dateien** oder ungleichmäßig verteilte Daten (Data Skew).



Über-Partitionierung: eine zu granulare Partitionierungsspalte erzeugt viele winzige Dateien pro Partition – das erhöht den Metadaten-Overhead und verlangsamt Lesezugriffe.

Trotzdem gibt es sinnvolle Anwendungsfälle für Partitionierung:

- **Isolation von Daten getrennter Schemata** (Multiplexing mehrerer Quellen in einer Tabelle).
- **DSGVO/CCPA-Löschanfragen**, bei denen typischerweise eine ganze Partition gelöscht werden soll (z. B. alle Daten eines bestimmten Tages).
- **SCD Type 2**, wo eine physische Trennung zwischen aktuellen und historischen Datensätzen die Performance verbessert.

Wird Partitionierung eingesetzt, gelten folgende Faustregeln: eine Spalte mit **niedriger Kardinalität** wählen, um nicht zu viele winzige Dateien zu erzeugen; jede Partition sollte zwischen 1 GB und 1 TB groß sein; besonders sinnvoll ist Partitionierung erst bei Tabellen, die auf über 1 TB anwachsen; üblicherweise wird nach einem Datum partitioniert. Z-Ordering lässt sich zusätzlich mit Partitionierung kombinieren, um Abfragen zu beschleunigen, die häufig auf eine zweite Spalte in der `WHERE`-Klausel filtern.

```
# Beispiel: Tabelle nach einer Spalte partitionieren
(df
  .write
  .mode('overwrite')
  .option("overwriteSchema", "true")
  .partitionBy('id') # Partitionierung nach id
  .saveAsTable("iot_data_partitioned")
)
```

Mit `DESCRIBE HISTORY` lässt sich die entstandene Partitionierung nachvollziehen – im `operationParameters`-Feld erscheint die gewählte Partitionierungsspalte, im `operationMetrics`-Feld die tatsächliche Anzahl erzeugter Dateien (bei zu hoher Kardinalität der Partitionierungsspalte kann diese Zahl schnell in die Tausende gehen, wie im obigen Beispiel mit `id` als Partitionierungsspalte). Mit `SHOW PARTITIONS <tabelle>` lassen sich zudem alle vorhandenen Partitionen einer Tabelle auflisten.

Ergänzt aus der offiziellen Databricks-Dokumentation: Die aktuelle Databricks-Dokumentation empfiehlt, für neue Tabellen standardmäßig **Liquid Clustering** zu verwenden und explizite Partitionierung nur noch für die oben genannten Spezialfälle (Compliance-Löschungen, sehr große Tabellen mit stabilen Zugriffsmustern) einzusetzen. Wird dennoch partitioniert, sollte die Partitionierungsspalte niedrige Kardinalität besitzen und regelmäßig mit `OPTIMIZE` kombiniert werden, um Small-File-Probleme zu vermeiden.

Quelle: [When to partition tables on Databricks – Databricks-Dokumentation](#)